

This database design follows the 3rd Normal Form (3NF) to ensure data integrity, performance, and auditability for the Banking Management System.

1. Database Overview

The schema is built to manage Users (Customers, Staff, Admins), their Accounts, financial Transactions, Loan workflows, and Audit trails. It supports the core business rules including fraud detection triggers and MFA security.

2. ER Diagram Description

- **Users** are the central entity linked to **Accounts**.
- **Accounts** generate **Transactions**.
- **Transactions** are monitored by the **Fraud_Logs** system.
- **Loans** are linked to **Users** and reviewed by **Employees**.
- **Audit_Logs** captures all sensitive actions across the system.

3. Tables & Schema

```
-- Core User Management
CREATE TABLE Users (
    user_id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    mfa_secret TEXT,
    role VARCHAR(20) CHECK (role IN ('CUSTOMER', 'EMPLOYEE', 'ADMIN')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Accounts (
    account_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES Users(user_id),
    account_type VARCHAR(50),
    balance DECIMAL(15, 2) DEFAULT 0.00,
    daily_limit DECIMAL(10, 2) DEFAULT 5000.00,
    status VARCHAR(20) DEFAULT 'ACTIVE'
);

-- Transactions & Security
CREATE TABLE Transactions (
    transaction_id SERIAL PRIMARY KEY,
    from_account_id INT REFERENCES Accounts(account_id),
    to_account_id INT REFERENCES Accounts(account_id),
    amount DECIMAL(15, 2) NOT NULL,
```

```

        transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        status VARCHAR(20) DEFAULT 'COMPLETED'
    );

CREATE TABLE Fraud_Logs (
    log_id SERIAL PRIMARY KEY,
    transaction_id INT REFERENCES Transactions(transaction_id),
    flagged_reason TEXT,
    status VARCHAR(20) DEFAULT 'PENDING' -- PENDING, CLEARED, BLOCKED
);

-- Loans
CREATE TABLE Loans (
    loan_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES Users(user_id),
    amount DECIMAL(15, 2) NOT NULL,
    credit_score_at_application INT,
    status VARCHAR(20) DEFAULT 'PENDING',
    approved_by INT REFERENCES Users(user_id),
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Budgeting
CREATE TABLE Budgets (
    budget_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES Users(user_id),
    category VARCHAR(50),
    limit_amount DECIMAL(10, 2),
    month_year DATE
);

-- Audit Trail (Mandatory Requirement)
CREATE TABLE Audit_Logs (
    audit_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES Users(user_id),
    action TEXT NOT NULL,
    table_name VARCHAR(50),
    record_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

4. Relationships

- ****One-to-Many:**** User -> Accounts (A customer can have multiple accounts).
- ****One-to-Many:**** User -> Loans (A customer can apply for multiple loans).
- ****One-to-Many:**** Account -> Transactions (An account records multiple transactions).
- ****One-to-One/Many:**** Transaction -> Fraud_Logs (A transaction might be flagged).

5. Indexes

- `CREATE INDEX idx\_user\_email ON Users(email);` (For MFA/Login speed).

- ``CREATE INDEX idx_trans_date ON Transactions(transaction_date);`` (For report generation).
- ``CREATE INDEX idx_loan_status ON Loans(status);`` (For Employee dashboards).
- ``CREATE INDEX idx_fraud_status ON Fraud_Logs(status);`` (For Admin monitoring).

6. Constraints

- **Check Constraint:** ``ALTER TABLE Transactions ADD CONSTRAINT check_amount_positive CHECK (amount > 0);``
- **Business Rule:** Trigger for transaction > \$10,000 to auto-populate ``Fraud_Logs``.
- **MFA enforcement:** Application level logic relies on the existence of ``mfa_secret`` in the ``Users`` table.

7. Normalization Notes

- **1NF:** All columns contain atomic values.
- **2NF:** All non-key attributes are fully functional dependent on the Primary Key.
- **3NF:** No transitive dependencies; for example, loan application data is separated from general User profile data, and Audit logs are centralized to ensure no modification of historical events.

8. Implementation Strategy

- **Security:** Ensure all data at rest is encrypted via Database TDE (Transparent Data Encryption).
- **Scalability:** Partition ``Transactions`` by ``transaction_date`` to handle historical growth efficiently.
- **Auditability:** Every update to the ``Loans`` table must trigger an ``INSERT`` into ``Audit_Logs`` using database triggers.